UNIVERSITY OF CAMBRIDGE

MPhil in Data Intensive Science

Easter Term 2026

SKA Minor Module Coursework
H. T. J. Bevins

*This coursework has two sections. Attempt **all** parts of the coursework.*

There are 100 marks in total. The anticipated number of marks allocated to each part of a question is indicated in the right margin, to be assessed from the code you provide and your report. Your report should contain all written answers from both sections, as well as any plots or images, as necessary.

This coursework should be submitted via a GitLab repository which will be created for you. Place all of your code, plots, and your report into this repository. The report should be in pdf format and placed in a folder named `report`*. You will be provided access to your repository until 23:59 on 1st April 2026, after which your work will be deemed ready to assess.*

Do not commit any data files you use into the repository. Instead, create a `data` *folder and inside that provide a text file listing the data files that would need to be present in that directory for your code to run.*

Similarly, do not commit any large temporary or output files you make into the repository. Instead, create an `outputs` *folder where any such files may be stored/accessed by your code.*

You are expected to submit code and associated material that demonstrates good software development practices as covered in the Research Computing module.

Your report should not exceed 3000 words (including tables, figure captions and appendices but excluding references); please indicate its word count on its front cover. You are reminded to comply with the requirements given in the Course Handbook regarding the use of, and declaration of use of, autogeneration tools.

© 2025 University of Cambridge

Version 1.0

One of the key science goals of the SKA is to observe the Cosmic Dawn, corresponding to around 0.5 - 2 billion years after the big bang when the first stars and galaxies formed, using the 21-cm signal from neutral hydrogen. By observing the fluctuations in the intensity of the 21-cm signal over cosmic time, researchers hope to learn when the first stars and galaxies formed, how bright they were, when the first black holes and exotic objects formed and when the universe transformed from being largely neutral material to ionized.

Instruments like the SKA aim to observe the spatial fluctuations in the 21-cm signal at different epochs in the form of a power spectrum. Since the intensity of the signal is dependent on light from the first stars, we expect to see more power on galactic scales than others at the corresponding redshifts. However, observations are complicated by non-trivial instrumental effects, and the instruments are also sensitive to emission from other radio sources including our own galaxy and radio stations!

To seperate the 21-cm signal from other radio sources and overcome instrumental challenges, researchers build detailed forward models of their data and use Bayesian inference techniques to learn about the early universe. In this course work, you will perform two different types of Machine Learning enhanced inference on mock observations of the 21-cm signal and compare their outputs.

To keep things simple, the mock observations here consist of a theoretical model of the 21-cm power spectrum and noise. The theoretical model of the signals used in this course work have been generated for you with the 'Zeus21' code[1] and are dependent on; the expansion rate of the universe h, the X-ray luminosity $L_{40}^{\text{X-ray}}$ of early galaxies, the escape fraction of ionized photons in galaxies of 10 solar masses f_{esc}^{10} and the star formation efficiency in galaxies of 10 solar masses ϵ. The goal is to recover the posterior on these parameters using a likelihood based inference pipeline with a neural network emulator and a simulation based inference pipeline that uses binary classification.

You can use PyTorch, TensorFlow, Flax or any other machine learning framework to complete this coursework. However, PyTorch and Flax are recommended.

Part 1 requires you to sample a likelihood function to recover a posterior distribution on the parameters of the 21-cm signal. Again, you can use any sampler you like to do this, however I recommend using emcee [2] or dynesty [3]. Both are relatively easy to use and are Python native, meaning they are easy to install!

You will need to download 'observations.npz' and 'simulations.zip' from the moodle. The 'observations.npz' file contains the mock data you will perform inference on (the k modes and power are included in the file). 'simulations.zip' contains approximately 10,000 power spectra with varying parameters that you can use to train the neural networks in question 1 and 2. Each spectrum is in a '.npz' file with the corresponding astrophysical parameters and cosmological parameters. All of the power spectra are at redshift 13 with k modes ranging from approximately 0.001 to 0.9 Mpc^{-1}.

You may also want to look ahead at lectures 15 and 16 of the course on moodle or

[1] `https://github.com/JulianBMunoz/Zeus21`

[2] `https://emcee.readthedocs.io/en/stable/`

[3] `https://dynesty.readthedocs.io/en/latest/index.html`

© 2025 University of Cambridge

github[4].

1 Likelihood Based Analysis of the 21-cm Power Spectrum

In Likelihood Based Inference methods researchers aim to describe the non-deterministic part of their observations with a probabilistic model. If we have a data set D and a model M then the difference between them $D - M$ should, assuming a perfect theoretical model, correspond to the noise in the observations. The goal is to then evaluate the likelihood or probability of the data given the model and parameters for many different combinations of parameters drawn from a prior to find the set that best describes the data. Likelihood based methods like this often end up calling the model 100,000s to millions of times and as a result the runtime of the model can become a significant computational bottleneck. Researchers therefore train neural network emulators to approximate the theoretical models to a high degree of accuracy in a fraction of the run time.

In this question, you will train an emulator on the power spectrum simulations from Zeus21 and fit the observations to recover a posterior distribution.

(a) In order to fit the power spectrum observations, you need to train a neural network emulator for the 21-cm power spectrum, and you need to preprocess the data for training. You should build a neural network that takes in the four model parameters $f_{\rm esc}^{10}$, $L_{40}^{\rm X-ray}$, ϵ_* and h that we want to do inference on and returns coefficients on the N most important PCA components to reconstruct the 21-cm signal (see Lecture 16). First split the data sets into testing, training and validation sets. Normalise the data sets appropriately and then perform a PCA decomposition on the training data and assess how many components are needed for an accurate reconstruction. *You should include a brief description of PCA in your report and describe how you assessed how many PCA components are needed. Include any relevant figures.*

[10]

(b) Determine an appropriate normalisation scheme. Then train your neural network emulator and validate its performance. *Include a description of your normalisation routine in your report and a graph of the distribution of training samples before and after normalizing. Include figures showing the performance of the network.*

[20]

(c) The likelihood function for the 21-cm power spectrum, assuming a perfect modelling and removal of the foregrounds, can be approximated by a Gaussian in

[4] `https://github.com/htjb/Talks/tree/master/Lectures/MPhil_Data_Intensive_Science_Lectures`

© 2025 University of Cambridge

the limit of many observations in each k bin (central limit theorem)

$$\ln L = -\frac{1}{2}\sum_k \frac{(P^{\rm obs}(k) - P^{\rm model}(k,\theta))^2}{\sigma^2(k)} - \frac{1}{2}\sum_k \log 2\pi\sigma^2(k). \tag{1}$$

where the noise σ depends both on the thermal noise and the sample variance

$$\sigma^2(k) = \frac{2(P^{\rm model}(k) - P^{\rm noise}(k))^2}{N_k} \tag{2}$$

where $P^{\rm noise}(k)$ is the noise power and N_k is the number of modes per k bin. We will assume 100 modes per k bin throughout the course work. A good model for the noise power is $P^{\rm noise}(k) = f^{\rm noise}P^{\rm model}(k)$ where $f^{\rm noise}$ can take a value between 10^{-3} and 10^0.

Using the emulator you have trained in the previous part of this question, perform Bayesian inference on the power spectrum in 'observations.npz' to determine the values of $f_{\rm esc}^{10}$, $L_{40}^{\rm X-ray}$, ϵ_* and h corresponding to the observations.

If you are using dynesty, infer the prior ranges for the astrophysical and cosmological parameters from the data in 'simulations.npz'.

Include a corner plot of the posterior in your report and describe what you can infer about properties of galaxies in the early universe from this data.

[20]

© 2025 University of Cambridge

2 Simulation Based Inference of the 21-cm Power Spectrum

In practice analytic likelihood functions like the one you encountered in the previous question might not describe the data well. An alternative approach to inference, that is becoming increasingly popular, is to try to learn the likelihood distribution with a neural network. This is called Simulation Based Inference. In this question, you will train a simulation based inference algorithm on the Zeus21 simulations and use it to perform inference on the mock observations.

We will continue to assume the simple Gaussian noise model we introduced in the previous question. In practice however one would build a detailed forward model of one's observations including simulations of the instrument and potential systematics that might not be easily expressed in an analytic likelihood function.

(a) There are a few different simulations based inference algorithms that work in slightly different ways. In this exercise, you need to build a neural ratio estimator (NREs) that takes in parameters and noisy power spectra and outputs the ratio of the likelihood to Bayesian evidence. *Describe how can we use an NRE to get a posterior distribution on the parameters of our model for a given observation in your report.*

[5]

(b) In order to effectively learn the likelihood function, the NRE needs to see simulations of observations that include instrumental effects, noise and systematics, not just the theoretical model of the power spectrum. Take the theoretical models in 'simulation.npz' and turn them into mock observations (theory plus noise) by drawing from the likelihood distribution in equation 1. For each theoretical model, use a different value of f^{noise} and record this alongside the corresponding astrophysical and cosmological parameters. *Include a figure in your report showing a handful of simulations before and after adding noise.*

[5]

(c) NREs are trained using the binary classification loss by being shown pairs of data and parameters drawn from the same distribution and pairs drawn from the disjoint distribution. Mathematically, the network learns to predict $r = \frac{P(D,\theta)}{P(D)P(\theta)}$. The loss is given by

$$L = \frac{1}{N}\sum_{i}^{N}\left[y_i \log(\sigma[\log(r(D,\theta))]) + (1-y_i)\log(1-\sigma[\log(r(D,\theta))])\right] \quad (3)$$

where σ is the sigmoid function and y_i is the label associated with data D and parameters θ. For our problem we therefore need pairs of data and parameters that go together e.g. $[D(\theta), \theta]$ with a label of $y_i = 1$ and pairs that do not e.g. $[D(\theta), \phi]$ with a label of $y_i = 0$.

From the last part you have a set of noisey simulations and their corresponding parameters $[f_{\text{esc}}^{10}, L_{40}^{\text{X-ray}}, \epsilon_*, h, f^{\text{noise}}]$. Appropriately label these simulations and use them to create a set of simulations from the disjoint distribution. *In your report*

© 2025 University of Cambridge

include a figure showing the distribution of power at $k = 0.1\ Mpc^{-1}$ *vs one of the four astrophysical parameters for the samples from the joint and disjoint distributions.* [5]

(d) Build and train an NRE to classify whether pairs of data and parameters go to together or not. It is common practice to have a linear activation function on the final layer of the NRE so that the network outputs $\log r$. The sigmoid is then applied inside the loss function during training. *In your report demonstrate the performance of your network on the test data set.* [20]

(e) Use the NRE and your answer to part (a) of this question to perform inference on 'observations.npz'. *In your report, include a posterior distribution from your inference and compare it with the distribution from question 1.*

[15]

END OF PAPER

© 2025 University of Cambridge